Chaos Engineering

chukmunnlee@nus.edu.sg

© Copyright National University of Singapore

Average Downtime Cost Per Hour by Industry

Industry	Cost per Hour	Key Risk Factor
Financial services	$5 - 7 million	Trading halts, transaction failures, regulatory fines
Healthcare	$1 - 2 million	Patient safety, HIPAA violations, care disruption
E-commerce/Retail	$100K - 1 million	Lost sales, cart abandonment, band damage
Manufacturing	$100K - 5 million	Production stoppage, supply chain disruption
SaaS/Technology	$100K - 5 million	SLA penalties, customer churns, refunds

Estimates from https://www.zmanda.com/blog/average-cost-of-downtime-enterprises/

© Copyright National University of Singapore

Modern Cloud Native Applications

- Micro services == distributed system
  - Complex, not "if" but "when" it will fail
- Traditional approach
  - Wait for failures to happen often in production
  - React to failures and fix it
  - Hope the patch will be effective in preventing future failures
- Better approach
  - Proactively and systematically discover system weaknesses
  - Build robustness through controlled experiments
  - Verify system resiliency before customer experience failures

© Copyright National University of Singapore

Chaos Engineering

- Practice of building and sustaining complex system by intentionally injecting faults into (near) production system in a controlled way
  - Not breaking things randomly but systematically and methodically
- Why - what you believe will happen vs what actually happened
  - Discover hidden architectural weaknesses
  - Validate automated recovery mechanism actually works
  - Improve observability and analysis skills in the engineering team
- Pioneered by Netflix around 2011 with their Chaos Monkey tool
  - Features - injects latency, randomly kills processes, fault tolerance validation, automated/scheduled runs
  - https://netflix.github.io/chaosmonkey/

© Copyright National University of Singapore

Goals and Objectives

- Proactive not reactive
  - Find the problem before they surface at the most unexpected and inconvenient time
- Systematic and scientific
  - Hypothesis driven experiments
  - Eg. the circuit breaker should transition to open when there is a maximum of 20 pending requests
- Production focus
  - Test the system where it matter most
- Controlled blast radius
  - Minimize failure risk during experiments while maximizing learning and discovery on system's behaviour

© Copyright National University of Singapore

Benefits

- Discover hidden system weaknesses
  - Undiscovered single point of failure
  - Failover mechanism that failed to trigger when needed
  - Cascading failure across microservices because of incorrect a circuit breaker configuration
- Build real confidence vs false sense of security
  - No errors found during testing does not mean it will not fail in production under real load
  - Discover monitoring gaps, cannot be alerted on metrics that are not measured
- Fill knowledge gaps
  - Discover how system behaves when it is under stress
  - Clarify and drill response procedures
- Impact on business
  - Estimate revenue loss from outages
  - Impact on compliance and SLA violations

© Copyright National University of Singapore

Environment for Chaos Engineering

- Application characteristics
  - Microservices - distributed applications with multiple dependencies
  - High availability - systems with strict SLAs, high data consistency
  - Rapid growth and scale - high volume, short latency and unpredictable traffic patterns
- Environment characteristics
  - Production is the gold standard, staging rarely mirrors the complexity, scale, traffic and 'noise' of production
  - Sufficient traffic to observe meaning patterns
  - Ability to sandbox/isolate blast radius e.g. canary deployments, a specific region
  - In place observability to monitor failures and capture system's behaviour

© Copyright National University of Singapore

Principles

- Define a steady state
  - Define normal system behaviour using key metrics like average response time, throughput
- Hypothesize failures
  - Predict behaviour when system fails or under stress
- Introduce fault into (near) production systems
  - Inject fault to system like network latency, process termination, etc.
- Observe system behaviour
  - Use observability tools to monitor system
- Automate and iterate
  - Continuously run chaos experiments to improve resilience

© Copyright National University of Singapore

Example - Principles

1. **Define steady state**

The service can process 100 checkout per minute with 5 replicas

2. **Hypothesize failure**

Assuming that there are 3 replicas of the service running, then we should be able to process approximately 60 checkouts per minute

3. **Inject fault**

Reduce the deployment to 3 replicas with an OOM fault

4. **Observe system behaviour**

Verify that the 3 replica service can sustain a 60 request checkouts per minute

5. **Prove or refute hypothesis**

Use the captured data validate the hypothesis

Image adapted from https://www.alibabacloud.com/blog/essential-for-reliability-assurance-performing-chaos-engineering-on-the-cloud_599399

© Copyright National University of Singapore

Chaos Engineering Toolkit

© Copyright National University of Singapore

DYI Simple Fault Injection

- Stop the database
  - `sudo systemctl stop mysql`
- DDoS your application
  - `siege -c 100 https://myapp.com`
- Max out your system resources
  - `stress-ng --cpu 8 --vm 4`
- Randomly drop packets on network interface
  - `tc qdisc add dev eth0 root netem loss 20%`
- Block access to DNS
  - `iptable -A INPUT -p tcp -m tcp --dport 53 -j DROP`

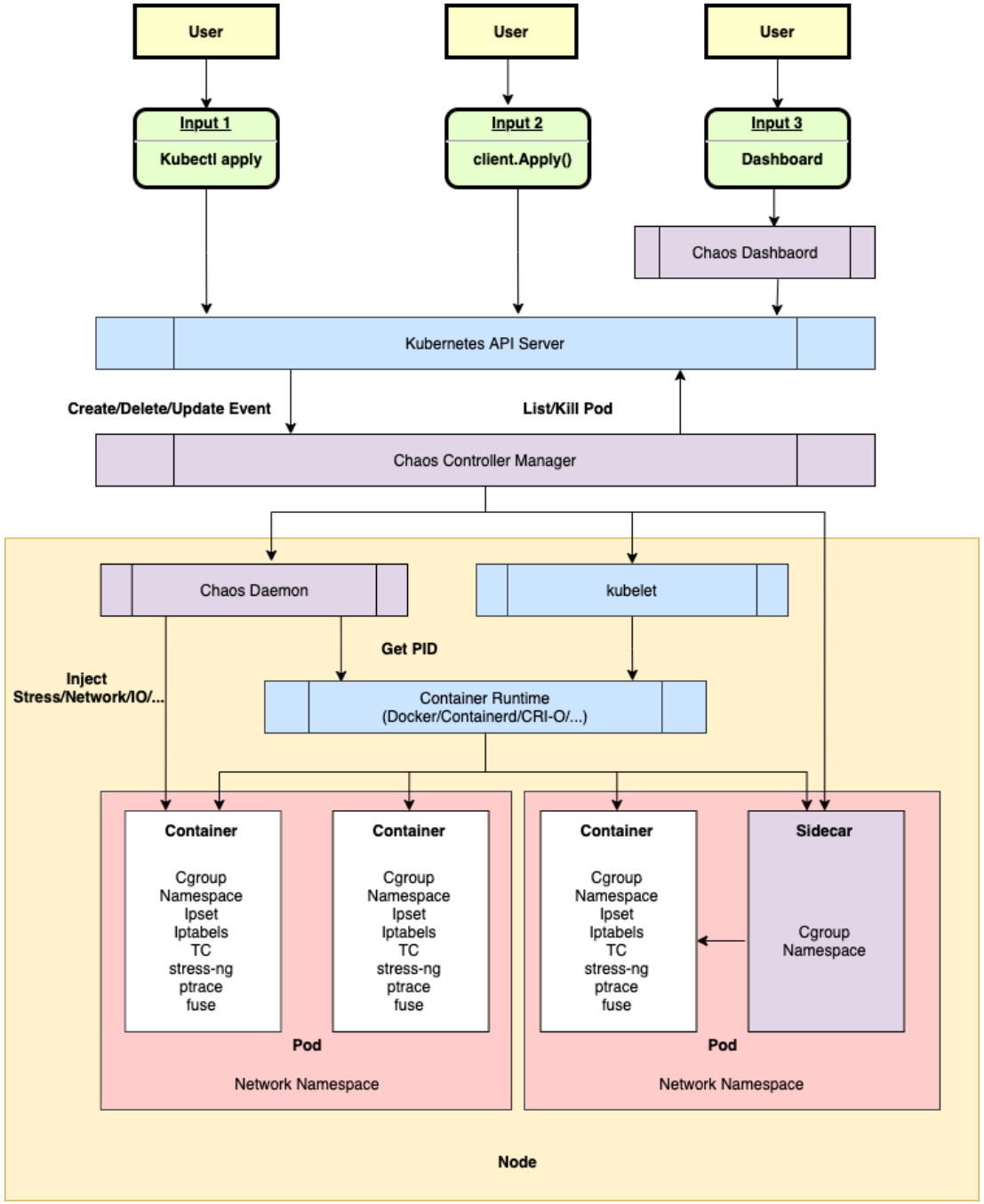
© Copyright National University of Singapore

Adapted from https://speakerdeck.com/adhorn/applying-chaos-engineering-principles-for-building-fault-tolerant-applications

Chaos Mesh

- Chaos engineering toolkit for Kubernetes
  - Installed via Helm charts
  - Main page - https://chaos-mesh.org
  - Open source - https://github.com/chaos-mesh/chaos-mesh
- Key features
  - Create chaos experiments with web UI and/or CRDs
  - Rich experiment types - network, pods, stress, time, kernel, nodes, DNS, etc
  - Experiments can be scheduled or weaved into a workflow to simulate complex faults
  - Safety mechanism includes
    - Namespace/RBAC isolation
    - Using labels to target specific pods
    - Experiment durations

© Copyright National University of Singapore



Architecture

- Chaos controller manager
  - Manages the lifecycle of chaos experiments
- Chaos daemon
  - Runs as **`DaemonSet`** on every node
  - Interfaces with the network, file system, kernel, etc
  - Performs the chaos operation on the node or selected pods running on the node
- Chaos dashboard
  - Web UI for designing, monitoring and managing experiments visually
- Admission webhook
  - Validates chaos CRDs for safety and correctness

Image from https://chaos-mesh.org/docs/

© Copyright National University of Singapore

Core Concepts

- Fault types are deployed as CRDs
  - Selectors - choose targets by namespace, labels, fields, annotations, etc.
  - Modes - how many targets are affected
    - One, all, fixed number, percentage
  - Duration - how long to run the experiments before stopping
  - Scheduling - Cron like scheduling
- DAG-based orchestration to combine multiple experiments
  - Can be run in serial or parallelly
  - Flow control to manage conditional execution, retries, timeouts, failures, etc.

© Copyright National University of Singapore

Chaos Mesh Web UI

Chaos Mesh

Dashboard
Workflows
Schedules
Experiments
Events
Archives
Settings
Documentation

All

Experiments
Schedules
Workflows
Total Experiments Status

Tutorial
Take a quick look at the functions of Chaos Mesh Dashboard
Tutorial

Newbie
Quickly inject a fault from a simple experiment
New experiment

Timeline

Recent Events

Enter the token (RBAC Authorization) to continue

Don't know how to get the token? Click here to generate

Name

chaos-mesh

Please fill the token name

Token

5ZGU2N2QtNGZIMy00NDM0LTg2NmMtZGJjYzg4ZmNjMDgyIiwia3ViZXJuZXRlcy5pbyI6eyJuYW1lc3BhY2UiOiJjaGFvcy1tZXNoIiwic2VydmljZWFjY291bnQiOnsibmFtZSI6ImNoYW9zLWRhc2hib2FyZCIsInVpZCI6IjdkYTY5MGJlLTBhNjctNDQzZS05ZDA5LTc5MmU1NGRiODU2NiJ9fSwibmJmIjoxNzcwNjE1MTM0LCJzdWIiOiJzeXN0ZW06c2VydmljZWFjY291bnQ6Y2hhb3MtbWVzaDpjaGFvcy1kYXNoYm9hcmQifQ.KA9roR1qZiMKvij0W_3wI43FWR457Ou8tCtA_5Dhe8EFa5FweDVR85TN4e4wXNhTFFa-67D0N5piWUgqMPPLUvK_V9nDyIWGdfJczfrf3usXYtUwX4zvpj88Wn77bPMBq0eNSCdz9TQbLwPk4T_o6zFC1RhTcRPOttdeNLMmP93vVcnZi7c49v9iK4cugbIL_Tp5d2oRoEqlZD0OoqVSFk48_bs7jEFLYnJk55BQqmSQQ5m75hA5ZQ_HUVDCjZcmYeOE3RRSwTfLh7brpZCSOGXIUvuSMZ1A2Ab-vVQn7y_4609W6QWR3IHGV8CP-a5Ri75OcXggjxREqIQbZ0T2Jw

Please fill the secret token

Submit

© Copyright National University of Singapore

Chaos Mesh Web UI

Chaos Mesh

Search

Choose namespace
All

- Dashboard
- Workflows
- Schedules
- Experiments
- Events
- Archives
- Settings
- Documentation

Experiments
0

Schedules
0

Workflows
0

Total Experiments Status

No experiments found

Tutorial
Take a quick look at the functions of Chaos Mesh Dashboard

Tutorial

Newbie
Quickly inject a fault from a simple experiment

New experiment

Start with workflow
Workflow allows you to organize multiple experiments to simulate more realistic failures

New Workflow

Start with schedule
Schedule allows you to run an experiment periodically

New Schedule

Timeline

No events found

Recent Events

No events found

© Copyright National University of Singapore

Chaos Experiments

© Copyright National University of Singapore

Chaos Experiments

Chaos	Description
DNSChaos	Simulates DNS failures by returning DNS queries, incorrect IP address, etc.
HTTPChaos	Simulates HTTP failure by intercepting HTTP requests and responses. It can abort connection, add latency, modify request/response or patch headers
IOChaos	Simulates file system faults such as IO delays, read/write errors, etc.
NetworkChaos	Simulates network failures including latency, packet loss/corruption/duplication, network partition
PodChaos	Simulate pod-level failures, including termination, unavailability, container kills. Useful for testing application resiliency to pod disruptions
StressChaos	Simulates CUP and memory stress to pods. Uses stress-ng to generate load
TimeChaos	Injects clock skew into pods to simulate time drifts. Useful to testing time-sensitive applications like distributed databases, caches, etc.
JVMChaos	Inject faults into JVM applications using bytecode manipulation.

© Copyright National University of Singapore

Example - **DNSChaos**

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: DNSChaos
metadata:
  name: dns-chaos
  namespace: prod
spec:
  action: random
  mode: all
  selector:
    labelSelectors:
      tier: frontend
  duration: 15m
  patterns:
  - ordersdb.ecom.svc.cluster.local
  - cart.prod.svc.cluster.local
```

Apply this chaos to all selected pods

Valid actions include **error** or **random**

Target all pods from all namespaces with the label **tier=frontend**

Run this experiment for 15 minutes

Apply this chaos to the following queries

See https://chaos-mesh.org/docs/simulate-dns-chaos-on-kubernetes/

© Copyright National University of Singapore

Example - **HTTPChaos**

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: HTTPChaos
metadata:
  name: http-delay-chaos
  namespace: prod
spec:
  mode: one
  selector:
    namespaces:
      - prod
  duration: 15m
  target: Request
  port: 8080
  method: GET
  path: /api/*
  delay: 5s
```

Apply this chaos randomly to one selected pod

Target all HTTP request in the prod namespace

Introduce a 5s latency to all **`GET /api/*`** request

See https://chaos-mesh.org/docs/simulate-http-chaos-on-kubernetes/

© Copyright National University of Singapore

Example - **HTTPChaos**

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: HTTPChaos
metadata:
  name: http-abort-chaos
  namespace: prod
spec:
  mode: fixed-percent
  value: 50
  selector:
    namespaces:
    - prod
  duration: 15m
  target: Response
  port: 8080
  method: POST
  path: /api/checkout
  replace:
    code: 403
```

Apply this chaos to 50% of the eligible requests

Target all HTTP request in the prod namespace

Deny checkout request

See https://chaos-mesh.org/docs/simulate-http-chaos-on-kubernetes/

© Copyright National University of Singapore

Example - IOChaos

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: IOChaos
metadata:
  name: io-latency-chaos
  namespace: prod
spec:
  action: fault
  mode: one
  selector:
    expressionSelectors:
    - { key: tier, operator: In, values: [ persistence ] }
  duration: 15m
  path: /var/lib/mysql/**/*
  errno: 5
  percent: 20
  methods: [ READ, WRITE ]
```

List of supported actions: **latency**, **fault**, **attrOverride**, **mistake**

Complex expression for targeting pods

Volume mount point

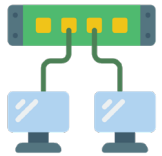
Specific Linux error number

Types of affected operations

See https://chaos-mesh.org/docs/simulate-io-chaos-on-kubernetes/

© Copyright National University of Singapore

Example - **NetworkChaos**



```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: network-latency-chaos
  namespace: prod
spec:
  action: delay
  mode: all
  selector:
    labelSelectors:
      tier: business
  duration: 15m
  delay:
    latency: 1000ms
    jitter: 250ms
    correlation: 75
```

List of supported actions: **delay**, **loss**, **duplicate**, **partition**, **corrupt**, **bandwidth**

Latency added to each packet

Random variation added to the latency 1000±250ms

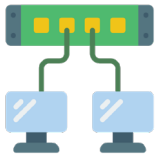
Controls the jitter from one packet to the next

⚠ Correlation is used to model burst or transient congestion. Eg. a 75% correlation means that there is a .75 probability that a jitter from a network packet will be applied to a following packet

See https://chaos-mesh.org/docs/simulate-network-chaos-on-kubernetes/

© Copyright National University of Singapore

Example - **NetworkChaos**



```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: network-partition-chaos
  namespace: prod
spec:
  action: partition
  mode: all
  selector:
    namespaces: [ prod ]
    fieldSelectors:
      'metadata.name': mysql-0
  duration: 15m
  direction: both
  target:
    namespaces: [ prod ]
    labelSelectors:
      role: replica
```

Block traffic in both directions

Simulate a 'split-brain' in a MySQL cluster by blocking network traffic from the master to the replicas

See https://chaos-mesh.org/docs/simulate-pod-chaos-on-kubernetes/

© Copyright National University of Singapore

Example - **PodChaos**

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: pod-chaos
  namespace: prod
spec:
  action: pod-failure
  mode: random-max-percent
  value: 75
  selector:
    labelSelectors:
      tier: business
  duration: 15m
```

List of supported actions: **pod-failure**, **pod-kill**, **container-kill**

See https://chaos-mesh.org/docs/simulate-dns-chaos-on-kubernetes/#configuration-description

© Copyright National University of Singapore

Example - StressChaos

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: StressChaos
metadata:
  name: pod-chaos
  namespace: prod
spec:
  mode: one
  selector:
    labelSelectors:
      tier: business
  containerNames: [ api-container ]
  duration: 15m
  stressors:
    memory:
      workers: 1
      size: 512MB
      time: 3min
    cpu:
      workers: 4
      load: 100
```

Optionally target specific containers of a selected pod

Occupy the containers memory until 512MB over 3 min

Increase the CPU utilization until 100%

See https://chaos-mesh.org/docs/simulate-heavy-stress-on-kubernetes/

© Copyright National University of Singapore

Example - **TimeChaos**

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: TimeChaos
metadata:
  name: time-shift-chaos
  namespace: prod
spec:
  mode: all
  selector:
    labelSelectors:
      tier: business
  duration: 15m
  timeoffset: -5m
```

Shift the real time clock back by 5 mins

⚠ Only affects the primary process of the container (pid 1) and all its child processes

See https://chaos-mesh.org/docs/simulate-time-chaos-on-kubernetes/

© Copyright National University of Singapore

Example - **JVMChaos**

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: JVMChaos
metadata:
  name: jvm-chaos
  namespace: prod
spec:
  action: latency
  mode: all
  selector:
    labelSelectors:
      app: payment
    containerNames: [ api-container ]
  duration: 15m
  class: com.example.PaymentService
  method: processPayment
  latency: 5000
```

Valid actions include **latency**, **return**, **exception**, **stress**, **gc**, **ruleData**

Introduce a 5sec latency to the **processPayment** method in **PaymentService** class

See https://chaos-mesh.org/docs/simulate-jvm-application-chaos/

© Copyright National University of Singapore

Example - **JVMChaos**

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: JVMChaos
metadata:
  name: jvm-chaos
  namespace: prod
spec:
  action: exception
  mode: fixed-precent
  value: 50
  selector:
    labelSelectors:
      app: payment
    containerNames: [ api-container ]
  duration: 15m
  class: com.example.UserService
  method: getUserById
  exception: 'IllegalArgumentException("Invalid user")'
```

Throw an exception 50% of the time when **getUserById** is invoked

See https://chaos-mesh.org/docs/simulate-jvm-application-chaos/

© Copyright National University of Singapore

Selector Summary

Namespace selector

Select all pods in the namespace

```
selector:
  namespaces:
  - stage
  - prod
```

Label selector

Select all pods with the label(s)

```
selector:
  labelSelectors:
    app.Kubernetes.io/name: payment
```

Annotation selector

Select all pods with the annotation(s)

```
selector:
  annotationSelector:
    sidecar.istio.io/inject: "yes"
```

Field selector

Select the pod(s) that matches the field

```
selector:
  fieldSelectors:
    'metadata.name': database
```

Expression selector

Select pod(s) that matches the defined set of expression

```
selector:
  expressionSelector:
  - { key: tier, operator: In, value: [cache] }
```

Pod list selector

Select a list of pods from the given namespace

```
selector:
  pods:
    prod-ns: # namespace
    - mysql-0
    - mysql-1
```

© Copyright National University of Singapore

Schedule

- Schedule chaos experiment with the **`Schedule`** CRD
- Scheduling rules
  - Cron based
    - Eg. 5 minutes - '`*/5 * * * *`'
  - With `@`
    - Eg. `@hourly` is equivalent to '`0 * * * *`'
    - Eg. `@weekly` is equivalent to '`0 0 * * 0`'
    - Eg. `@every 5m` is equivalent to '`*/5 * * * *`'

© Copyright National University of Singapore

Example - **Schedule**

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: Schedule
metadata:
  name: schedule-jvm-chaos
  namespace: prod
spec:
  schedule: '@every 2h'
  type: JVMChaos
  jvmChaos:
    action: latency
    mode: all
    duration: 15m
    ...
```

Schedule a **JVMChaos** to run every 2 hours for 15 minutes

Chaos CRD name

Use camel case chaos experiment CRD name for configuration

See https://chaos-mesh.org/docs/define-scheduling-rules

© Copyright National University of Singapore

Orchestrating Chaos Experiments

© Copyright National University of Singapore

Workflow

- Chaos experiments specifies what chaos experiments to perform
  - Eg. disrupt the network, kill the pod
- Workflow defines how and when multiple experiment occurs
  - Manage a group of experiments
  - Created with the **`Workflow`** CRD
- Workflows allow you to create complex chaos scenarios
  - Control the timing and sequencing of chaos events
  - Includes 'health checks' in workflow to stop experiments if system starts to collapse

© Copyright National University of Singapore

Workflow Concepts

- Workflow
  - Defines an orchestration of the chaos experiments
- Template
  - A DAG node in a workflow
  - Represents an operation eg. chaos experiment, serial/parallel grouping, etc.
- Entry
  - The first template in the workflow
- Templates
  - **`Serial`** - run child templates consecutively
  - **`Parallel`** - run child templates concurrently
  - **`Suspend`** - pause the workflow for a specified duration
  - **`StatusCheck`** - performs a HTTP health check
  - Specific chaos types - eg. **`PodChaos`**, **`NeworkChaos`**, **`StressChaos`**, etc

© Copyright National University of Singapore

Example - Serial Workflow

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: Workflow
metadata:
  name: serial-workflow
  namespace: prod
spec:
  entry: main
  templates:
  - name: main
    templateType: Serial
    deadline: 30m
    children:
    - stress
    - pause
    - network-delay
```

Template to start the workflow

Template must complete by this duration; otherwise the workflow will fail

Run these templates sequentially

```yaml
- name: stress
  templateType: StressChaos
  deadline: 10m
  stressChaos:
    ...
```

Configuration for **StressChaos**, not shown

```yaml
- name: pause
  templateType: Suspend
  deadline: 1m
```

Suspend for 1 minute

```yaml
- name: network-delay
  templateType: NetworkChaos
  deadline: 10m
  networkChaos:
    ...
```

A template consist of a name, deadline to constraint how long the template will run, the template type and its specific configuration

© Copyright National University of Singapore

Example - Parallel Workflow

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: Workflow
metadata:
  name: serial-workflow
  namespace: prod
spec:
  entry: main
  templates:
  - name: main
    templateType: Parallel
    deadline: 15m
    children:
    - stress
    - network-delay
```

Run these templates in parallel

```yaml
  - name: stress
    templateType: StressChaos
    deadline: 10m
    stressChaos:
      ...

  - name: network-delay
    templateType: NetworkChaos
    deadline: 10m
    networkChaos:
      ...
```

© Copyright National University of Singapore

Status Check

- Perform health checks on the system during chaos experiment
  - Only support HTTP checks
- Mode
  - Continuous - keep checking during the duration of the deadline until the check fails or deadline is exceeded
  - Synchronous - only check once
- Success criterial is defined by the returned status code
  - Eg 200 - if endpoint returns a 200 status code
  - Eg. 200 - 399 - if endpoints return status code from 200 to 399 (inclusive)
- **`abortWithStatusCheck`** - if true, then abort the workflow

© Copyright National University of Singapore

Example - Workflow with Status Check

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: Workflow
metadata:
  name: serial-workflow
  namespace: prod
spec:
  entry: main
  templates:
  - name: main
    templateType: Parallel
    deadline: 20m
    children:
    - io-failure
    - health-check
```

Run health check in parallel with the chaos experiment. Continuously monitor the services for actual degradation.

```yaml
- name: io-failure
  templateType: IOChaos
  deadline: 15m
  ioChaos:
    ...

- name: health-check
  templateType: StatusCheck
  deadline: 15m
  abortWithStatusCheck: true
  statusCheck:
    mode: Continuous
    intervalSeconds: 15
    timeoutSeconds: 5
    failureThreshold: 3
    successThreshold: 1
    http:
      url: http://catalogue.prod.svc...
      method: GET
      criteria:
        statusCode: 200-299
```

Safety! Abort workflow if check fails

Run the health check for the duration of the experiment

Run the health check until either it has failed or the duration has exceeded the deadline. Or **Synchronous**

Check is successful if request returns status code 200 - 299

© Copyright National University of Singapore

Example - Workflow with Schedule

```yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: Workflow
metadata:
  name: serial-workflow
  namespace: prod
spec:
  entry: main
  templates:
  - name: main
    templateType: Parallel
    deadline: 20m
    children:
    - network-partition
    - health-check
```

Schedule network chaos to run every minute for 30 seconds

```yaml
- name: network-partition
  templateType: Schedule
  deadline: 15m
  schedule:
    schedule: '@every 1m'
    concurrencyPolicy: Forbid
    type: NetworkChaos
    networkChaos:
      action: partition
      duration: 30s
      ...
```

Prevent multiple network chaos to run concurrently. Alternative is **`Allow`**.

```yaml
- name: health-check
  templateType: StatusCheck
  deadline: 15m
  abortWithStatusCheck: true
  statusCheck:
    mode: Continuous
    ...
```

© Copyright National University of Singapore

Observations from Chaos Experiment - 1

- Steady state / SLO
  - Any SLO breach that is not expected or contained
  - Eg. Error rate, latency percentiles, throughput
- Blast radius
  - Eg. which users, regions that felt the fault
- Fault detection
  - Eg. did any configured alerts fired
- Recovery mechanism
  - Validate the in-place contingencies are working
  - Eg. did the circuit breaker trip, are the entries in the service discovery updated correctly

© Copyright National University of Singapore

Observations from Chaos Experiment - 2

- Affected dependencies
  - Eg. what are the 'unrelated' services that are affected by the outage?
  - Faulty services should be isolated
- Observability gap
  - No visibility into the fault
  - Eg. missing traces, logs, metrics to discover what is happening
- Data integrity and corruption
  - Eg. duplicate payment, dropped orders
- Human response
  - Eg. is the SOP adequate to manage and contain the fault

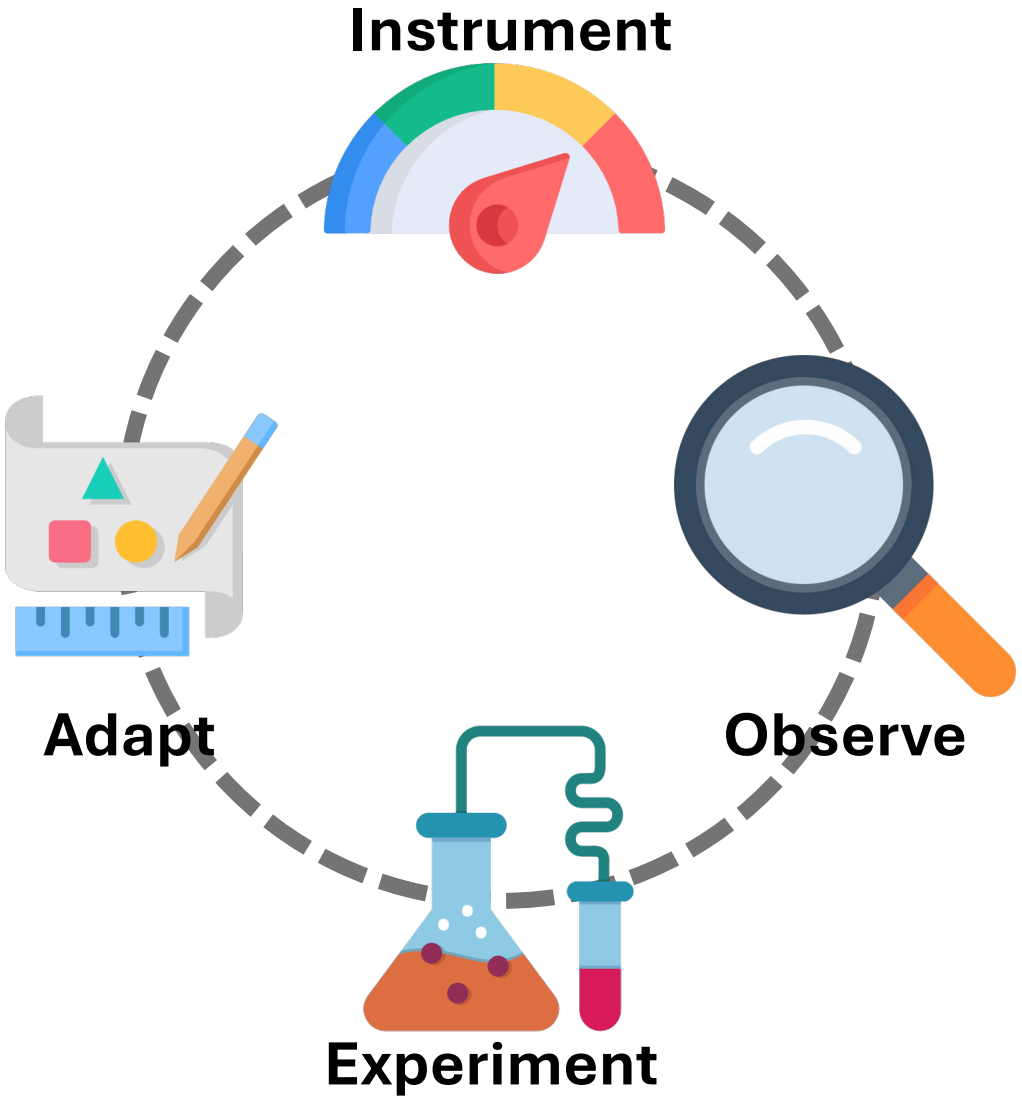
© Copyright National University of Singapore

Pre-requisite for Chaos Engineering

- Observability as foundation
  - Implemented comprehensive metrics, logs and traces
  - Created dashboards and alters to monitor systems
  - Ability to correlate signals across services
- Resiliency patterns
  - Implemented circuit breakers, auto-scaling, rate limiting, service discovery, etc
  - Graceful degradation capabilities
- Deployment automation
  - Adopted infrastructure as code
  - Ability to rollback to last stable release
- Maturity
  - Defined SLO and SLI
- Executive support
  - Budget for tooling and dedicated time
  - Acceptance of controlled risks
- Blameless culture
  - Safety to experiment
  - Learn from failures without punishment
  - Shared responsibility for resilience and reliability

© Copyright National University of Singapore

Virtuous Cycle - Resilience through Observability



- Instrument - build telemetry into application and infrastructure
- Observe - correlate signals across services to observe and analyse behaviours
- Experiment - apply chaos engineering practices to introduce failures to study how system behaves under stress
- Adapt - use gained insights to strengthen resilience posture

© Copyright National University of Singapore

Observability is the prerequisite for chaos engineering,
and chaos engineering is the best way to test your observability

© Copyright National University of Singapore

End of Module

© Copyright National University of Singapore